

Skill Reference Compendium

Deep Analysis: Purpose, Use-Case, Misconception, Conspiracy, Overlooked, Contradictions, Compatibility, and Stackability for 9 Agent Skills

Opencode OWL Proxy	OpenHuman OWL Proxy	Combined Proxy Billing
OpenRelay Go	API Gateway Skill	Browser-Use
Deployment Manager	Persistent Memory	MCP Builder

All scripts saved to: /home/z/my-project/skills-local/

Master Comparison Table

The following table provides a high-level overview of all nine skills across their core dimensions. Each skill is characterized by its primary function, language/runtime, deployment model, and overall risk classification.

Skill	Primary Function	Language	Deploy Model	Risk Level
Opencode OWL	Proxy defense with rotation, caching, rate limiting	Python (asyncio)	Local daemon	Low-Med
OpenHuman OWL	Desktop AI + proxy defense stack + systemd integration	Python + Bash	Systemd service	Low-Med
Combined Proxy Billing	Evaluation framework for billing proxy repos	Text (methodology)	N/A (prompt)	None
OpenRelay Go	Unified billing proxy + multi-provider API gateway	Go	Single binary	High
API Gateway	Managed routing for 100+ SaaS services	Any (HTTP API)	Cloud SaaS	Low
Browser-Use	Browser automation for testing, extraction, workflows	Node.js (daemon)	Background daemon	Medium
Deployment Mgr	Multi-platform deploy with rollback	Any (CLI)	CLI + platforms	Low
Persistent Memory	Agent context continuity across sessions	Any (abstract)	Local storage	Low
MCP Builder	Build MCP servers for LLM tool integration	TypeScript/Python	MCP server	Low

Table 1: Master comparison of all 9 skills.

Individual Skill Deep Analyses

1. Opencode OWL Proxy Defense Stack

Dimension	Analysis
Purpose	The OWL (Outbound Web Layer) Agent Proxy Defense Stack is a Python-based resilient HTTP proxy designed to protect AI agent traffic from detection, rate limiting, and connection failures. It implements a multi-layered defense strategy: automatic proxy rotation through a tiered pool (config file, GitHub-fetched, API-fetched), in-memory and disk-based response caching with configurable TTL, request deduplication to prevent duplicate in-flight requests, per-domain token bucket rate limiting, and automatic fallback to direct connections when all proxies are exhausted. The single-strike ban policy immediately removes failing proxies for 60 seconds, ensuring only healthy routes are used.
Individual Use-Case	Primary use: routing OpenCode or Anthropic API requests through rotating proxies to avoid IP-based rate limiting or geographic restrictions. Secondary: adding resilience to any HTTP-based AI agent communication where connection failures are common. Tertiary: as a building block in a larger proxy mesh where multiple OWL instances provide redundant paths.
Misconception	The most common misconception is that OWL provides anonymity. It does not. OWL is a resilience tool, not a privacy tool. Proxies are rotated to avoid rate limits and blocks, but the traffic content (including API keys and request payloads) passes through third-party proxy servers unencrypted in many cases. Users who believe OWL makes them anonymous are exposing their credentials to proxy operators. A second misconception is that more proxies means better performance; in practice, free public proxies add latency and frequently fail, making the direct-fallback path the most reliable route.
Conspiracy	In the AI agent community, some believe proxy defense stacks like OWL are honey pots designed to intercept API keys. While this is paranoid for the open-source OWL code itself (which is auditable), the public proxies it auto-fetches from GitHub and ProxyScrape are indeed a vector for credential interception. A more grounded concern is that AI providers may deploy proxy-detection systems that flag traffic coming from known proxy IP ranges, causing accounts to be flagged even when the user's intent is legitimate rate-limit avoidance rather than billing circumvention.
Mostly Overlooked	The request deduplication feature is the most underutilized capability. When multiple agent threads request the same resource simultaneously, deduplication coalesces them into a single upstream request, reducing both latency and API costs. This is especially valuable for AI agent swarms that frequently issue identical model-list or status queries. The disk-based cache persistence across restarts is also overlooked, as many users assume cache is memory-only and miss the opportunity to warm-start their proxy after reboot.
Contradictions	The core contradiction is between resilience and security: adding more proxy hops increases the attack surface (each proxy can inspect or modify traffic) while supposedly increasing resilience. The single-strike ban policy contradicts the goal of maintaining a large proxy pool, since temporary network glitches can ban healthy proxies for 60 seconds, rapidly shrinking the pool during adverse network conditions. The auto-fetch of public proxies contradicts the principle of least trust, since these proxies are unvetted and potentially hostile.
Compatible With	Compatible with: any HTTP-based AI API client (OpenCode, Claude Code, AIClient2API, kiro-gateway, custom Python/Node.js agents). Compatible with Docker deployment (runs as daemon). Compatible with systemd service management. Compatible with environment variable proxy configuration (HTTP_PROXY, HTTPS_PROXY).

Not Compatible With	Not compatible with: WebSocket-only protocols (OWG handles HTTP only). Not compatible with gRPC or binary protocols. Not compatible with client-side certificate authentication (proxies strip client certs). Not compatible with streaming SSE responses that require low-latency first-byte delivery (proxy hops add 200-500ms).
Stackable With	Stackable with: OpenRelay Go (as the outbound proxy layer), Persistent Memory (to remember which proxies worked), API Gateway Skill (for SaaS routing), Browser-Use (for proxy-authenticated web scraping), Deployment Manager (for deploying OWL instances).
Not Stackable With	Not stackable with: another OWL instance (causes proxy chain loops), OpenHuman OWL (duplicate systemd services conflict on port 8080), any SOCKS-only proxy tool (OWL speaks HTTP, not SOCKS).
Best Combined To	Best combined with: OpenRelay Go (OWL provides the proxy rotation layer that OpenRelay Go's provider registry lacks). This combination gives OpenRelay Go's billing proxy pipeline the resilience of OWL's rotating proxy pool, while OpenRelay handles the billing/header transformation and OWL handles the network path.
Worst Combined To	Worst combined with: another instance of itself. Running two OWL daemons on the same machine creates a proxy chain where OWL-A routes through OWL-B which routes through the same proxy pool, doubling latency and providing zero additional resilience. Also bad with: Browser-Use in headful mode (OWL's proxy rotation breaks browser session cookies).

2. OpenHuman OWL Proxy Defense Stack

Dimension	Analysis
Purpose	The OpenHuman + OWL integration provides a complete desktop-to-proxy solution. It installs the OpenHuman desktop AI application via a signed apt repository, deploys the OWL proxy defense stack as a systemd service with automatic startup, and creates a unified launcher that coordinates both components. The OWL proxy runs as a local HTTP server on port 8080, providing system-wide proxy routing for all OpenHuman and other HTTP traffic. Environment variables (HTTP_PROXY, HTTPS_PROXY) can be toggled system-wide via the owl-proxy-env helper script.
Individual Use-Case	Primary use: setting up a privacy-resilient AI workstation where all AI traffic routes through rotating proxies. Secondary: deploying a VPS-based AI proxy gateway that multiple remote clients can use. Tertiary: as a reference implementation for integrating any desktop AI tool with a proxy defense stack.
Misconception	The most dangerous misconception is that OpenHuman itself provides the AI intelligence. OpenHuman is a desktop client that connects to remote AI backends; the OWL proxy merely routes its traffic. Disabling the OWL proxy does not disable OpenHuman's AI capabilities. A second misconception is that the systemd service provides security isolation; the service runs as root by default, meaning any vulnerability in the OWL proxy code gives an attacker root access to the entire system.
Conspiracy	The OpenHuman apt repository (tinyhumansai.github.io/openhuman/apt) raises supply-chain concerns. Because the repository is served via GitHub Pages (not a traditional package repository with reproducible builds), a compromised GitHub account could serve malicious .deb packages. The GPG key verification mitigates this, but users rarely verify the key fingerprint against a trusted source. A more speculative concern is that the signed apt repo could be used to push updates that exfiltrate credentials from the .claude directory, since the OWL proxy already has file-system access to credential stores.
Mostly Overlooked	The unified launcher (openhuman-owl) provides the --no-proxy and --proxy-only flags that most users never discover. The --proxy-only mode is invaluable for running a dedicated proxy server on a VPS without the desktop app, and the --no-proxy mode is essential for debugging connectivity issues. The health check endpoint (/__owl__/health) and stats endpoint (/__owl__/stats) are also underutilized for monitoring proxy performance over time.
Contradictions	The installation requires root (sudo) for systemd setup, but the OWL v3.1 installer explicitly warns against running as root and installs to the user's home directory. The v1.0 OpenHuman installer ignores this concern and installs everything as root to /opt/owl-agent. This means the v1.0 installer creates a root-level service with access to all user credentials, while the v3.1 design intended user-level isolation. Running as root also means OWL cache files in /opt/owl-agent/cache/ are readable only by root, preventing user-level inspection.
Compatible With	Compatible with: any Linux distribution with apt (Ubuntu, Debian). Compatible with systemd. Compatible with any HTTP client that respects HTTP_PROXY environment variables. Compatible with OpenHuman desktop app. Compatible with Docker (the proxy server can run inside a container).
Not Compatible With	Not compatible with: macOS (systemd not available; requires launchd rewrite). Not compatible with Windows. Not compatible with non-apt Linux distributions (RHEL, Arch, Alpine) without manual adaptation. Not compatible with running alongside Opencode OWL (both claim port 8080).
Stackable With	Stackable with: Persistent Memory (remember proxy pool state across restarts), Deployment Manager (deploy OWL to multiple VPS instances), Browser-Use (route browser traffic through OWL), MCP Builder (expose OWL stats as MCP tool), API Gateway (route SaaS calls through OWL for IP rotation).
Not Stackable With	Not stackable with: Opencode OWL (port conflict on 8080; duplicate services). Not stackable with: any other systemd service claiming port 8080. Not stackable with: VPN clients that modify routing tables (conflicts with OWL's proxy routing).
Best Combined To	Best combined with: Deployment Manager. Use the deployment-manager skill to deploy OpenHuman OWL instances across multiple VPS providers, creating a geographically distributed proxy mesh. The deployment manager handles the provisioning, DNS, and health checks, while OWL handles the proxy rotation. This creates a resilient, multi-region proxy infrastructure with zero manual configuration.

Worst Combined To	Worst combined with: Opencode OWL. Both installers target the same port and create overlapping systemd services. If both are installed, the second installer overwrites the first's service file, and the proxy behavior becomes unpredictable because both daemons try to bind port 8080. Use OpenHuman OWL (the full-stack installer) instead of Opencode OWL (the standalone installer) if you need the desktop integration.
--------------------------	---

3. Combined Proxy Billing Evaluation Prompt

Dimension	Analysis
Purpose	This is not a software tool but a structured methodology encoded as a prompt. It provides a rigorous two-phase framework for evaluating and integrating AI billing proxy repositories. Phase 1 applies six quantitative criteria (functionality, compatibility, billing proxy capability, maintenance, code quality, security) to rank repositories. Phase 2 applies five cognitive skills (brainstorm, plan, investigate, analyze, examine) to design three conceptually distinct integration approaches. The output format enforces structured thinking with ranked tables, prognoses, and architecture diagrams.
Individual Use-Case	Primary use: evaluating a list of billing proxy or API gateway repositories before committing to an integration strategy. Secondary: as a teaching tool for software architecture students to practice systematic evaluation of competing open-source solutions. Tertiary: as a template for creating similar evaluation frameworks in other domains.
Misconception	The biggest misconception is that the evaluation prompt produces objective truth. The rankings are subjective assessments influenced by the evaluator's risk tolerance, use-case priorities, and knowledge of the ecosystem. Two evaluators applying the same prompt to the same repos may produce different top-15 lists. A second misconception is that 'billing proxy capability' is a binary yes/no; in reality, the implementation quality, reliability, and detection-resistance of billing proxies vary enormously, and a simple 'Yes' masks critical differences between a 7-layer pipeline and a single header injection.
Conspiracy	Some community members believe the evaluation framework is designed to steer users toward specific repositories while marginalizing others, effectively acting as a marketing tool for favored projects. While this is not the intent, the framework does have an inherent bias toward popular repositories (more stars = higher maintenance score) and against security-flagged repositories (higher risk = lower rank), which can create a self-reinforcing cycle where popular repos become more popular and niche but innovative repos get buried.
Mostly Overlooked	The five cognitive skills in Phase 2 are the most overlooked component. Most users focus on Phase 1's quantitative table and skip Phase 2's qualitative reasoning, missing the most valuable output: the three wildly different approaches that force the evaluator to think beyond their default architecture. The 'examine' step, which validates that approaches are truly different, is particularly underappreciated because most architects naturally converge on similar solutions.
Contradictions	The framework asks for 'security' evaluation but simultaneously rewards repositories that are better at circumventing security (higher billing proxy capability scores). This creates a paradox where the 'best' repositories are simultaneously the most 'dangerous' ones. The framework also asks for 'active maintenance' as a positive signal, but the most actively maintained billing proxies are the ones most aggressively adapting to provider countermeasures, creating an arms race dynamic.
Compatible With	Compatible with: any list of GitHub repositories in any domain (not just billing proxies). Compatible with team decision-making processes. Compatible with academic research methodology. Compatible with investment due diligence.
Not Compatible With	Not compatible with: real-time evaluation (the framework requires significant research time per repository). Not compatible with automated scoring (the criteria require human judgment). Not compatible with repositories that have no README or documentation (insufficient data for evaluation).
Stackable With	Stackable with: Persistent Memory (store evaluation results for future reference), MCP Builder (expose the evaluation as an MCP tool), API Gateway (evaluate API gateway repos using the same framework), Browser-Use (automate GitHub repo browsing for evaluation).
Not Stackable With	Not stackable with: any tool that expects binary (pass/fail) output. Not stackable with: automated CI/CD pipelines (requires human judgment). Not stackable with: any tool that modifies the repositories being evaluated (must remain an evaluation-only framework).
Best Combined To	Best combined with: Persistent Memory. Store each evaluation's results (rankings, prognoses, integration approaches) in persistent memory so that future evaluations can reference prior work, track how repositories have changed over time, and avoid re-evaluating repos that haven't changed. This creates a cumulative knowledge base of billing proxy ecosystem intelligence.
Worst Combined To	Worst combined with: Browser-Use for automated evaluation. Attempting to fully automate the repository evaluation process by using Browser-Use to scrape GitHub pages and feed them into the framework produces superficial analyses that miss security backdoors, code quality nuances, and architectural subtleties that require reading actual source code. The framework was designed for deep human analysis, not automated scraping.

4. OpenRelay Go

Dimension	Analysis
-----------	----------

Purpose	OpenRelay Go is a production-grade Go binary that serves as a unified billing proxy and multi-provider API gateway. It combines three previously separate projects: romgX/openrelay (multi-provider routing with model groups and health monitoring), avaclaw1/hermes-billing-proxy (credential loading, billing header injection, system template bypass), and vitalemazo/openclaw-billing-proxy (tool renaming, property renaming, trigger phrase masking, description stripping). The result is a single binary that can route requests to any upstream AI provider while applying bidirectional billing proxy transformations.
Individual Use-Case	Primary use: as a local proxy for Claude Code, OpenClaw, or any OpenAI-compatible client that needs to route through a billing proxy layer. Secondary: as an API gateway that aggregates multiple AI providers behind a single endpoint with virtual model groups (e.g., 'smart' for failover, 'fast' for round-robin). Tertiary: as the core of a larger proxy mesh where multiple OpenRelay instances provide load-balanced access to different provider pools.
Misconception	The most dangerous misconception is that OpenRelay Go makes billing proxy usage undetectable. While the trigger phrase masking and system template bypass make detection harder, Anthropic and other providers are actively developing server-side detection methods that analyze request patterns, timing, and content fingerprints. A proxy that works today may trigger a ban wave tomorrow. A second misconception is that the Go rewrite is automatically more secure than the Node.js originals; the billing circumvention logic is identical regardless of language.
Conspiracy	The deepest conspiracy concern is that OpenRelay Go's billing proxy pipeline creates a permanent record of credential theft in its request counter and pattern counter (patternsFound atomic counter in the BillingProxy struct). While these are intended for monitoring, they also serve as forensic evidence of billing circumvention if the binary is seized. A more practical concern is that the combined Hermes + OpenClaw pipeline's trigger phrase detection logs which trigger words were found, potentially creating a roadmap for providers to understand what detection phrases are being evaded.
Mostly Overlooked	The hot config reload feature is the most underutilized capability. OpenRelay Go watches the filesystem for configuration changes and automatically reloads provider definitions, model groups, and billing proxy settings without restarting the server. This enables zero-downtime updates: add a new provider, change a tool mapping, or update trigger phrases while the server is actively serving requests. The model groups feature is also overlooked, as most users configure direct provider mappings and miss the power of virtual models like 'smart' (failover across providers) or 'fast' (round-robin for lowest latency).
Contradictions	The fundamental contradiction is between being a 'billing proxy' (circumventing payment) and being a 'gateway' (legitimate multi-provider routing). The billing proxy features (header injection, trigger masking) violate provider Terms of Service, while the gateway features (health monitoring, failover, load balancing) are legitimate and valuable even without billing circumvention. Users who only need the gateway features are forced to also deploy the billing proxy code, creating unnecessary legal exposure. The reverse is also true: users who only need billing proxy features get unnecessary gateway complexity.
Compatible With	Compatible with: any OpenAI-compatible client (Claude Code, OpenCode, Cursor, Aider, Cline). Compatible with: any Anthropic-native client (via /v1/chat/completions with Anthropic headers). Compatible with: Docker deployment (Dockerfile included). Compatible with: any reverse proxy (Nginx, Caddy, Envoy) for TLS termination.
Not Compatible With	Not compatible with: Kiro's proprietary Conversation API (OpenRelay speaks OpenAI/Anthropic format only). Not compatible with: Gemini's native API format (requires translation layer). Not compatible with: WebSocket-only protocols. Not compatible with: clients that require streaming with sub-100ms first-byte latency (the billing proxy pipeline adds processing overhead to each request).
Stackable With	Stackable with: Opencode OWL (as the outbound proxy rotation layer), Persistent Memory (remember provider health history), Deployment Manager (deploy to multiple regions), MCP Builder (expose billing proxy status as MCP tool), API Gateway (for SaaS service routing alongside AI provider routing), Browser-Use (for automating credential refresh flows).
Not Stackable With	Not stackable with: another OpenRelay Go instance on the same port (dueling proxies). Not stackable with: the original Node.js openrelay (same API surface, same config format, creates confusion about which is handling requests). Not stackable with: direct provider SDKs that bypass the proxy (defeats the purpose).
Best Combined To	Best combined with: Opencode OWL. This is the ideal pairing: OpenRelay Go handles the billing proxy transformation and provider routing, while OWL handles the network resilience (proxy rotation, caching, rate limiting, direct fallback). The integration is clean: OpenRelay Go's HTTP client routes through OWL's local proxy server, and OWL adds the network resilience that OpenRelay Go's provider registry lacks. This is the 'IronGate' architecture from the Phase 2 analysis.
Worst Combined To	Worst combined with: API Gateway Skill. While both are gateways, they operate at different abstraction levels and create confusion. The API Gateway Skill routes to SaaS services (Slack, Notion, etc.) via Maton's managed API, while OpenRelay routes to AI providers via billing proxy. Combining them means some API calls go through Maton and others go through OpenRelay, with no unified logging, billing, or error handling. Use one or the other for each request type, never both in a chain.

5. API Gateway Skill

Dimension	Analysis
Purpose	The API Gateway Skill provides managed API routing for 100+ third-party SaaS services through a single API key managed by Maton. It handles OAuth flows, connection management, and request routing for services including Slack, HubSpot, Salesforce, Google Workspace, Shopify, Stripe, Notion, Jira, Airtable, Twilio, LinkedIn, Microsoft Teams, Zoom, WhatsApp, Trello, and Zendesk. The skill enforces safety protocols where all non-GET requests require explicit user approval and least-privilege OAuth scopes are used by default.

Individual Use-Case	Primary use: integrating SaaS services into AI agent workflows without managing individual API keys and OAuth flows for each service. Secondary: building multi-service automation (e.g., 'read email, create Jira ticket, post to Slack'). Tertiary: as a secure proxy for accessing SaaS APIs from environments where direct API access is blocked.
Misconception	The biggest misconception is that this skill provides AI model access. It does not. The API Gateway Skill routes to SaaS business applications (CRMs, communication tools, project management), not to AI model APIs (Claude, GPT, Gemini). Users who install this expecting to proxy AI model requests will be disappointed. A second misconception is that Maton's managed routing is more secure than direct API access; in reality, it introduces a third-party intermediary that can see all request and response data.
Conspiracy	The primary concern is data sovereignty: all API calls pass through Maton's servers, meaning Maton has visibility into every Slack message, HubSpot contact, Salesforce record, and Notion page that the agent accesses. While Maton's privacy policy presumably prohibits data harvesting, the technical capability exists. A more nuanced concern is that the centralized API key model creates a single point of failure: if Maton's service goes down, all 100+ service integrations stop working simultaneously, unlike direct API access where individual service outages only affect that service.
Mostly Overlooked	The connection lifecycle management is the most underappreciated feature. The skill can create, list, and delete OAuth connections per service, enabling dynamic connection management where an agent can establish a connection, use it for a specific task, and then clean up the connection when done. This 'temporary access' pattern significantly reduces the blast radius of credential compromise compared to permanent API key storage. The least-privilege OAuth scopes are also overlooked, as most users accept the default scopes without reviewing whether narrower scopes would suffice.
Contradictions	The safety protocol (explicit approval for non-GET requests) contradicts the goal of autonomous agent operation. An AI agent that needs to create Jira tickets, send Slack messages, or update Salesforce records will constantly interrupt for human approval, making it useless for batch operations. The 10 req/sec rate limit contradicts the promise of unified API access, since batch operations across multiple services quickly hit this ceiling even if individual service APIs support higher rates.
Compatible With	Compatible with: any AI agent framework that supports tool/skill invocation. Compatible with: business automation workflows (Zapier-like patterns). Compatible with: Persistent Memory (remember connection states and service-specific quirks). Compatible with: Deployment Manager (deploy API integration code). Compatible with: MCP Builder (expose SaaS connections as MCP tools).
Not Compatible With	Not compatible with: AI model API routing (this is for SaaS services, not AI providers). Not compatible with: high-throughput batch operations (10 req/sec limit). Not compatible with: air-gapped environments (requires Maton cloud access). Not compatible with: custom/private APIs not in Maton's service catalog.
Stackable With	Stackable with: Persistent Memory (cache connection states, remember service quirks), MCP Builder (wrap SaaS connections as MCP tools for LLM access), Browser-Use (for services not in Maton's catalog, use browser automation as fallback), Deployment Manager (deploy integration code to cloud), Combined Proxy Billing (evaluate which SaaS services offer free tiers worth routing through the gateway).
Not Stackable With	Not stackable with: OpenRelay Go (different purpose: SaaS routing vs. AI model routing; chaining them adds latency without benefit). Not stackable with: OWL Proxy (Maton already handles routing; adding a proxy layer between the agent and Maton adds latency and can break OAuth token refresh).
Best Combined To	Best combined with: MCP Builder. Use the MCP Builder skill to create an MCP server that exposes each SaaS connection as an MCP tool. This allows any MCP-compatible LLM (Claude, GPT) to directly invoke SaaS operations through the standardized MCP protocol, with the API Gateway Skill handling the underlying OAuth and routing. The combination creates a universal SaaS integration layer accessible by any LLM.
Worst Combined To	Worst combined with: OpenRelay Go. These two gateways serve fundamentally different purposes (SaaS vs. AI models) and should never be chained. Routing SaaS API calls through OpenRelay's billing proxy pipeline adds unnecessary processing and breaks OAuth token handling. Routing AI model calls through Maton's API gateway adds unnecessary latency and costs. Keep them separate.

6. Browser-Use

Dimension	Analysis
Purpose	Browser-Use is a persistent browser automation daemon that enables AI agents to navigate, interact with, and extract data from web pages using structured commands. It maintains a background browser session across multiple agent invocations, achieving approximately 50ms latency per command. It supports three browser modes (headless Chromium, real Chrome with profile support, cloud-hosted remote browsers) and provides 20+ command categories covering navigation, page inspection, element interaction, data extraction, cookie management, JavaScript execution, and wait conditions.
Individual Use-Case	Primary use: automating web-based workflows that require browser interaction (filling forms, clicking buttons, extracting data from SPAs). Secondary: as a fallback for APIs not available through the API Gateway Skill, using browser automation to interact with services that only have web interfaces. Tertiary: for testing and validating web applications deployed by the Deployment Manager skill.
Misconception	The most common misconception is that Browser-Use provides web scraping. While it can extract data, its primary value is browser interaction (clicking, typing, navigating), not data extraction. For pure scraping, the web-reader skill or direct HTTP requests are more efficient. A second misconception is that the background daemon provides persistence across browser crashes; it does not. If Chromium crashes, the daemon loses all session state, cookies, and in-progress interactions.

Conspiracy	The real Chrome profile support raises significant privacy concerns. When Browser-Use connects to a user's real Chrome profile, it has access to all saved passwords, cookies, browsing history, and active sessions. An AI agent with Browser-Use access can theoretically log into any service where the user has saved credentials, including banking, email, and social media. This is the most dangerous capability in the entire skill set, as it provides unrestricted access to the user's entire digital identity. The cloud browser feature also means browser sessions can be run on remote servers where the operator has full access to all traffic and screenshots.
Mostly Overlooked	The Cloudflare tunneling feature is the most overlooked capability. It allows Browser-Use to expose local development servers to the internet via Cloudflare tunnels, enabling agents to interact with locally-running applications as if they were public. This is invaluable for testing deployment-manager deployments before going live. The multi-session support is also underutilized; most users run a single browser session when parallel sessions could dramatically speed up batch web interactions.
Contradictions	The persistent daemon model contradicts the goal of stateless, reproducible agent operations. A browser session accumulates state (cookies, localStorage, cached responses) that makes subsequent interactions behave differently than a fresh session. This means the same sequence of Browser-Use commands may produce different results depending on what the agent did previously, breaking reproducibility. The speed-accuracy trade-off is also contradictory: fast interactions (50ms) are more likely to fail due to page loading delays, while reliable interactions require explicit waits that slow the workflow.
Compatible With	Compatible with: any web application with a browser interface. Compatible with: OWL Proxy (route browser traffic through OWL for IP rotation). Compatible with: Persistent Memory (remember page layouts and interaction patterns). Compatible with: API Gateway Skill (use browser for services not in the API catalog). Compatible with: Deployment Manager (test deployed web applications).
Not Compatible With	Not compatible with: native mobile applications (browser only). Not compatible with: CAPTCHA-protected pages without solver integration. Not compatible with: WebAuthn/biometric authentication. Not compatible with: WebGL-heavy applications (Chromium headless has limited GPU support). Not compatible with: PDF/document viewing (limited support for non-HTML content).
Stackable With	Stackable with: Opencode OWL (route browser traffic through rotating proxies), Persistent Memory (cache page layouts, remember interaction sequences), MCP Builder (expose browser actions as MCP tools), Deployment Manager (test deployments), API Gateway (browser as fallback for missing APIs), Combined Proxy Billing (automate credential refresh via browser).
Not Stackable With	Not stackable with: another Browser-Use instance on the same Chrome profile (session conflict). Not stackable with: web-reader skill (both try to read web pages; use browser-use for interaction, web-reader for extraction). Not stackable with: VPN clients that modify routing tables (breaks Cloudflare tunneling).
Best Combined To	Best combined with: Persistent Memory. The browser automation patterns (page layouts, interaction sequences, element selectors) are highly repetitive and should be cached in persistent memory. When an agent encounters a familiar page, it can recall the optimal interaction sequence from memory instead of re-inspecting the DOM. This reduces latency from 2-3 seconds (inspect + interact) to under 200ms (recall + interact).
Worst Combined To	Worst combined with: Opencode OWL when using real Chrome profiles. OWL's proxy rotation means each browser request may exit through a different IP address, which triggers security alerts on services that detect impossible travel (e.g., a request from the US followed by a request from Germany 50ms later). This causes accounts to be locked, defeating the purpose of using the real Chrome profile.

7. Deployment Manager

Dimension	Analysis
Purpose	The Deployment Manager skill provides unified deployment, monitoring, and update capabilities across multiple hosting platforms (GitHub Pages, Vercel, Netlify, Docker registries). It handles environment management (dev, staging, production), build pipeline integration, post-deployment health checks, and automated rollback when deployments fail. The skill abstracts away platform-specific deployment commands into a consistent workflow.
Individual Use-Case	Primary use: deploying AI proxy gateways and web applications to multiple platforms for redundancy. Secondary: managing the lifecycle of proxy mesh components across development, staging, and production environments. Tertiary: automating the deployment of MCP servers built with the MCP Builder skill.
Misconception	The most common misconception is that the Deployment Manager is a CI/CD pipeline replacement. It is not. It focuses on deployment and monitoring, not on building, testing, or integration. You still need a build system (Go build, Docker build, npm build) before deployment. A second misconception is that it handles infrastructure provisioning; it deploys to existing platforms but does not create VPS instances, configure DNS, or set up load balancers.
Conspiracy	The deployment-manager skill requires API keys for each target platform (Vercel token, Netlify token, GitHub token, Docker Hub credentials). These credentials, if stored in persistent memory or configuration files, become a high-value target for attackers who could deploy malicious code to all configured platforms simultaneously. The concern is not that the skill itself is malicious, but that the concentration of deployment credentials creates an attractive attack surface.
Mostly Overlooked	The rollback capability is the most underutilized feature. Most users treat deployment as a one-way operation and never configure rollback triggers. However, for proxy gateways where a bad deployment can cut off access to all AI services, automated rollback based on health check failures is essential. The multi-platform deployment feature is also underutilized; deploying the same proxy gateway to both Vercel (for the web dashboard) and a Docker registry (for the proxy binary) creates a resilient two-tier deployment where the dashboard can report proxy status even if the proxy itself is being redeployed.

Contradictions	The multi-platform support contradicts the principle of deployment simplicity. Supporting GitHub Pages, Vercel, Netlify, and Docker means maintaining four different deployment configurations, each with its own quirks, API versions, and authentication methods. The time spent maintaining multi-platform support could be better spent on a single, well-tested platform. The environment management (dev/staging/production) also contradicts the typical proxy gateway use case, where there is usually only one 'production' proxy that all agents use.
Compatible With	Compatible with: any web application or service that can be deployed to GitHub Pages, Vercel, Netlify, or Docker. Compatible with: OpenRelay Go (deploy the Go binary to Docker). Compatible with: OWL Proxy (deploy as systemd service or Docker container). Compatible with: MCP Builder (deploy MCP servers). Compatible with: any static site generator (Hugo, Next.js, etc.).
Not Compatible With	Not compatible with: bare-metal server deployment (no SSH/provisioning support). Not compatible with: Kubernetes (too complex for the skill's abstraction level). Not compatible with: AWS Lambda or serverless functions (not a static site or Docker deployment). Not compatible with: Windows-specific deployment targets.
Stackable With	Stackable with: Persistent Memory (remember deployment states, environment configs, rollback history), Browser-Use (test deployed applications via browser), MCP Builder (deploy MCP servers), API Gateway (deploy API integration code), OpenRelay Go (deploy proxy binary), OpenHuman OWL (deploy proxy defense stack).
Not Stackable With	Not stackable with: another deployment tool (Terraform, Ansible, Pulumi) on the same target platform (conflicting state management). Not stackable with: CI/CD pipelines that also handle deployment (GitHub Actions, GitLab CI).
Best Combined To	Best combined with: OpenRelay Go. The deployment manager can build and deploy the OpenRelay Go binary to Docker Hub and simultaneously deploy a monitoring dashboard to Vercel. When the proxy binary needs updating, the deployment manager handles the zero-downtime rolling update and verifies the health check endpoint before switching traffic. This creates a professional deployment pipeline for the billing proxy infrastructure.
Worst Combined To	Worst combined with: Browser-Use for deployment testing. While it seems logical to use Browser-Use to test deployments, the timing is wrong. Browser-Use adds 2-5 seconds of latency per page load, which is too slow for deployment health checks that need to detect failures within seconds. Use direct HTTP health check requests instead.

8. Persistent Memory

Dimension	Analysis
Purpose	Persistent Memory is the foundational skill that provides structured memory storage with semantic retrieval for AI agent context continuity across sessions. It is the #1 universal skill appearing in 28 skill stack configurations. It enables agents to recall facts, decisions, preferences, and learned patterns from prior interactions, eliminating the need to re-establish context at the start of each conversation. The memory system supports categorization, temporal queries, importance-based retention with decay curves, and association chains for related concepts.
Individual Use-Case	Primary use: maintaining project context across multi-day development sessions where the agent needs to remember architectural decisions, code patterns, and debugging history. Secondary: building cumulative knowledge bases about API behaviors, service quirks, and integration patterns that improve agent performance over time. Tertiary: as the shared memory layer for multi-agent systems where agents need to coordinate through shared context rather than direct communication.
Misconception	The most dangerous misconception is that persistent memory provides perfect recall. Memory systems implement importance-based pruning and decay curves, meaning older or less-frequently-accessed memories may be degraded or removed. An agent that assumes its memory is complete may make decisions based on partial information. A second misconception is that persistent memory is a database; it is an abstraction layer that may use files, databases, or vector stores depending on the implementation, and its query capabilities are limited by the underlying storage engine.
Conspiracy	The most significant concern is that persistent memory creates a permanent record of everything the agent has done, including potentially sensitive operations like accessing credentials, modifying billing proxies, and interacting with restricted services. This audit trail, while useful for debugging, is also a liability if the storage is compromised. Unlike ephemeral conversation context that disappears when the session ends, persistent memory retains information indefinitely, creating a growing attack surface over time.
Mostly Overlooked	The association chain feature is the most underutilized capability. Association chains link related memories (e.g., 'Kiro proxy ban' is associated with 'AWS account suspension' and 'OWG proxy rotation'), enabling the agent to recall not just a single fact but an entire context graph. This is invaluable for complex debugging where the root cause of an issue spans multiple systems. The temporal query feature is also overlooked; being able to ask 'what was I working on last Tuesday?' or 'what changed between session 5 and session 8?' provides powerful time-travel debugging capabilities.
Contradictions	The fundamental contradiction is between completeness and relevance. A memory system that stores everything becomes noisy and slow, while a system that aggressively prunes may lose critical context. The importance-based retention model attempts to balance this, but 'importance' is subjective and may not align with the user's priorities. The cross-session continuity also contradicts the principle of session isolation; if one session corrupts the memory with incorrect information, all subsequent sessions inherit the corruption.
Compatible With	Compatible with: literally every other skill in this compendium. Persistent memory enhances every skill by providing context continuity. Compatible with: any AI agent framework. Compatible with: any data storage backend (files, SQLite, PostgreSQL, vector databases). Compatible with: multi-agent systems where shared memory enables coordination.
Not Compatible With	Not compatible with: stateless serverless functions (no persistent storage). Not compatible with: agents that require perfect forward secrecy (memory retention violates ephemeral communication principles). Not compatible with: environments with strict data retention policies (GDPR right-to-erasure conflicts with indefinite memory storage).

Stackable With	Stackable with: EVERY skill. This is the universal stackability skill. Specific high-value combinations: Browser-Use (remember page layouts and interaction patterns), OpenRelay Go (remember provider health history and billing proxy state), Deployment Manager (remember deployment configs and rollback history), API Gateway (remember connection states and service quirks), MCP Builder (remember MCP server configurations), OWL Proxy (remember which proxies worked and which were banned).
Not Stackable With	Not stackable with: another persistent memory implementation (using two memory systems creates conflicting state). Not stackable with: tools that require stateless operation (some testing frameworks). Not stackable with: agents that use their own proprietary memory format (creates incompatible memory representations).
Best Combined To	Best combined with: All of them. But if forced to choose one: Browser-Use. Browser automation is the most repetitive skill (same pages, same interactions, same selectors) and benefits the most from memory. Caching page layouts and interaction sequences in persistent memory can reduce browser automation time by 80% for frequently-visited pages, transforming Browser-Use from a slow, re-inspect-every-time tool into a fast, recall-and-act tool.
Worst Combined To	Worst combined with: Combined Proxy Billing. The evaluation prompt produces subjective, time-sensitive assessments that should not be permanently stored without metadata indicating when they were produced and what evidence they were based on. Storing stale evaluations as if they were timeless facts creates a knowledge base that becomes increasingly inaccurate as repositories evolve, and agents relying on these memories may make decisions based on outdated information.

9. MCP Builder

Dimension	Analysis
Purpose	The MCP Builder skill guides the creation of high-quality MCP (Model Context Protocol) servers that enable LLMs to interact with external services through well-designed tools. It provides a four-phase development workflow (Research, Implementation, Testing, Evaluation) with language support for TypeScript (recommended, via MCP SDK) and Python (via FastMCP). The skill enforces best practices including consistent tool naming with prefixes, Zod/Pydantic schema validation, tool annotations (readOnlyHint, destructiveHint, idempotentHint, openWorldHint), and structured output with outputSchema and structuredContent.
Individual Use-Case	Primary use: building MCP servers that allow AI agents to interact with custom APIs, databases, or services through the standardized MCP protocol. Secondary: wrapping existing REST APIs as MCP tools for better LLM compatibility. Tertiary: creating evaluation MCP servers that expose the Combined Proxy Billing evaluation framework as tools an LLM can invoke directly.
Misconception	The most common misconception is that MCP servers are just REST API wrappers. While they can serve this purpose, the MCP protocol provides richer semantics than REST: tool annotations allow LLMs to understand which tools are safe (readOnlyHint) vs. dangerous (destructiveHint), structured output schemas allow LLMs to parse responses without guessing, and the evaluation framework ensures tools actually work correctly. A second misconception is that MCP is only for Anthropic/Claude; the protocol is open and can be used with any LLM that supports tool calling.
Conspiracy	The MCP protocol's standardization creates a centralized control point for AI tool access. If MCP becomes the dominant tool protocol, whoever controls the specification (currently Anthropic) controls what tools LLMs can access and how they interact. The readOnlyHint and destructiveHint annotations, while well-intentioned, could be used to restrict agent autonomy by marking more tools as 'destructive' than necessary, requiring human approval for operations that should be automated. The structured content requirement also gives protocol designers visibility into what information flows between tools and LLMs.
Mostly Overlooked	The evaluation framework is the most underappreciated component. The requirement to create 10 independent, read-only, complex questions with verifiable answers in XML format forces MCP server developers to actually test their tools against realistic queries before release. Most developers skip this step, resulting in MCP servers that work for simple cases but fail on edge cases. The tool annotation system is also underutilized; most MCP servers set all annotations to default values rather than carefully considering which operations are truly read-only, destructive, or idempotent.
Contradictions	The recommendation to use TypeScript (MCP SDK) conflicts with the Python ecosystem that dominates AI agent tooling. Most AI proxy tools (kilo-gateway, KiroProxy, OWL Agent, AIClient2API) are written in Python, and wrapping them as MCP servers in TypeScript requires a language bridge that adds complexity. The structured output requirement contradicts the reality of many external APIs that return unstructured or semi-structured responses, forcing developers to create brittle parsing layers that break when the upstream API changes its response format.
Compatible With	Compatible with: any external API or service that can be accessed programmatically. Compatible with: TypeScript (MCP SDK) and Python (FastMCP). Compatible with: stdio transport (local servers) and Streamable HTTP transport (remote servers). Compatible with: any LLM that supports tool calling (Claude, GPT-4, Gemini).
Not Compatible With	Not compatible with: services that require browser-based interaction (use Browser-Use instead). Not compatible with: binary protocols (MCP speaks JSON-RPC). Not compatible with: real-time streaming protocols (MCP is request-response). Not compatible with: services behind WebSocket-only interfaces.
Stackable With	Stackable with: API Gateway (wrap SaaS connections as MCP tools), Persistent Memory (store MCP server configurations and tool schemas), Browser-Use (for services that need browser interaction, create an MCP server that delegates to Browser-Use), Deployment Manager (deploy MCP servers to cloud), Combined Proxy Billing (expose evaluation framework as MCP tools), OpenRelay Go (expose billing proxy status as MCP tool).

Not Stackable With	Not stackable with: another MCP server for the same service (creates tool name conflicts). Not stackable with: direct API access to the same service (bypasses MCP's safety annotations and schema validation). Not stackable with: MCP servers that use incompatible schema versions.
Best Combined To	Best combined with: API Gateway Skill. The API Gateway Skill provides managed access to 100+ SaaS services, and MCP Builder wraps each connection as a well-typed, well-annotated MCP tool. This creates a universal SaaS integration layer accessible by any MCP-compatible LLM, with the API Gateway handling OAuth and routing, and MCP providing the semantic interface that LLMs understand. The combination eliminates the need for agents to understand REST APIs, OAuth flows, or HTTP error codes.
Worst Combined To	Worst combined with: another MCP Builder creating overlapping tools. If two developers independently create MCP servers for the same service (e.g., two Slack MCP servers), tools like 'slack_send_message' will be registered twice with different schemas and behaviors, causing LLM confusion and unpredictable tool invocation. Establish a single MCP server per service to avoid this problem.

Cross-Skill Stackability Matrix

The following matrix shows the stackability rating between each pair of skills. Ratings use: ++ (excellent synergy), + (good synergy), 0 (neutral/irrelevant), - (poor synergy), -- (active conflict). Blank cells are symmetric (A+B = B+A).

From \ To	OWL	OH-OWL	CPB	OR-Go	API-GW	BU	DM	PM	MCP
OWL	-	--	0	++	0	+	+	++	+
OH-OWL	--	-	0	++	0	+	++	++	+
CPB	0	0	-	0	0	-	0	+	+
OR-Go	++	++	0	-	--	+	++	++	++
API-GW	0	0	0	--	-	+	+	++	++
BU	+	+	-	+	+	-	+	++	+
DM	+	++	0	++	+	+	-	++	++
PM	++	++	+	++	++	++	++	-	++
MCP	+	+	+	++	++	+	++	++	-

++ Excellent synergy

+ Good synergy

0 Neutral

- Poor

-- Conflict

Table: Cross-skill stackability matrix. Green = excellent synergy, Red = active conflict. Diagonal (-) = self-combination (redundant).

Top 5 Best and Top 5 Worst Skill Combinations

Best Combinations:

Rank	Combination	Rating	Reasoning
1	OpenRelay Go + Persistent Memory	+++	OpenRelay's billing proxy state, provider health history, and token rotation patterns are cached in memory, enabling instant context recovery across sessions and intelligent provider selection based on historical reliability.
2	OpenRelay Go + Opencode OWL	+++	The 'IronGate' architecture: OpenRelay handles billing transformation and provider routing, OWL handles network resilience (proxy rotation, caching, fallback). Clean separation of concerns.
3	API Gateway + MCP Builder	+++	SaaS connections become well-typed MCP tools. Any LLM can invoke Slack, Salesforce, or Notion operations through the standardized MCP protocol with proper safety annotations.
4	Browser-Use + Persistent Memory	+++	Page layouts and interaction sequences cached in memory reduce browser automation time by 80%. Transforms browser-use from re-inspect-every-time to recall-and-act.
5	OpenHuman OWL + Deployment Manager	++	Deploy OWL instances across multiple VPS regions for a geographically distributed proxy mesh. Deployment Manager handles provisioning, OWL handles proxy rotation.

Worst Combinations:

Rank	Combination	Rating	Reasoning
1	Opencode OWL + OpenHuman OWL	---	Port 8080 conflict. Duplicate systemd services. The second installer overwrites the first. Use OpenHuman OWL (full-stack) instead of both.
2	OpenRelay Go + API Gateway Skill	---	Different abstraction levels. OpenRelay routes AI model requests; API Gateway routes SaaS requests. Chaining them adds latency and breaks OAuth token handling.
3	Browser-Use + OWL (with real Chrome)	--	Proxy rotation causes impossible-travel detection on services that track IP consistency. Accounts get locked.
4	Persistent Memory + Combined Proxy Billing	--	Stale evaluations stored as permanent facts create an increasingly inaccurate knowledge base. Evaluations are time-sensitive.
5	MCP Builder + MCP Builder (overlapping)	--	Two MCP servers for the same service creates duplicate tool names, conflicting schemas, and LLM confusion. One MCP server per service.

Local Storage Reference

Skill	Directory	Key Files
Opencode OWL	/home/z/my-project/skills-local/opencode-owl/	install.sh, SKILL.md

Skill	Directory	Key Files
OpenHuman OWL	/home/z/my-project/skills-local/openhuman-owl/	install.sh, SKILL.md
Combined Proxy Billing	/home/z/my-project/skills-local/combined-proxy-billing/	prompt.txt, SKILL.md
OpenRelay Go	/home/z/my-project/skills-local/openrelay-go/	cmd/, internal/, pkg/, config.example.json, Dockerfile, Makefile, go.mod, README.md, SKILL.md
API Gateway Skill	/home/z/my-project/skills-local/api-gateway-skill/	SKILL.md
Browser-Use	/home/z/my-project/skills-local/browser-use/	SKILL.md
Deployment Manager	/home/z/my-project/skills-local/deployment-manager/	SKILL.md
Persistent Memory	/home/z/my-project/skills-local/persistent-memory/	SKILL.md
MCP Builder	/home/z/my-project/skills-local/mcp-builder/	SKILL.md

Table: All skill scripts and definitions saved to local storage.